

Contents

guide DAPO: An Open-Source LLM Reinforcement Learning System at Scale

0. 这篇论文到底在改写什么

1. 论文结构地图

2. 背景: R1 之后, 大家卡在哪里

3. 先分清 PPO、GRPO、DAPO

4. DAPO 总流程

5. Rule-based reward 和答案整数化

6. DAPO objective 的张量形状

7. Trick 1: Clip-Higher

8. Trick 2: Dynamic Sampling

9. Trick 3: Token-Level Policy Gradient Loss

10. Trick 4: Overlong Reward Shaping

11. 四个 trick 的证据链

12. Training details: 这些数字要记住

13. 训练动态: DAPO 让你监控什么

14. Case study: 反思行为不是硬编码的

15. DAPO 和 PRM/RLVR 的关系

16. 和后续 VAPO、Dr.GRPO、GenRM 的关系

17. 本仓库代码地图

18. 30-60 分钟本地实验

19. AI agent 应该怎样辅助你学 DAPO

20. 常见误读

21. 现代意义

22. 闭卷掌握检查

guide_DAPO: An Open-Source LLM Reinforcement Learning System at Scale

原论文: DAPO: An Open-Source LLM Reinforcement Learning System at Scale

本地原文 PDF: [learning/rl-sota-2026/paper/01_dapo.pdf](#)

作者: Yu et al., ByteDance Seed, AIR/Tsinghua, HKU

年份: 2025

类型: paper

0. 这篇论文到底在改写什么

DAPO 不是一篇 “我提出一个全新 RL 算法” 的论文，而是一篇把 reasoning LLM 大规模 RL recipe 公开拆开的论文。它的核心价值是: 在 DeepSeek-R1、OpenAI o1 等 reasoning model 把 RLVR 和 test-time scaling 推到台前之后，社区知道 “RL 能让 base model 长出复杂推理”，但不知道大规模训练到底有哪些关键细节。DAPO 把这些细节整理成四个可复现的技术点，并开源算法、训练代码和数据。

论文的结果很直接: 用 Qwen2.5-32B base model 做 RL，在 AIME 2024 上达到 50 分，超过论文中引用的 DeepSeek-R1-Zero-Qwen-32B 的 47 分，并且使用约一半训练步数。作者说他们一开始用 naive GRPO 只能到 30 分，说明真正的难点不是 “会写 GRPO 公式”，而是让大模型长 CoT RL 稳定跑起来。

你读这篇论文时要抓住四个病灶和四个药方:

1. Entropy collapse: 采样变得越来越确定，探索空间塌缩。药方是 Clip-Higher。
2. Zero-gradient prompts: 一组样本全对或全错时 group advantage 为 0。药方是 Dynamic Sampling。
3. Long-CoT loss imbalance: 每条 response 同权会扭曲 token 的训练权重。药方是 Token-Level Policy Gradient Loss。
4. Truncation reward noise: 长答案被截断后直接给惩罚会把正确推理当错惩罚。药方是 Overlong Reward Shaping。

这篇论文适合接在 DeepSeek-R1、PPO、GRPO、PRM/RLVR 后面读。前面几篇告诉你“为什么要用 RL 和可验证奖励”，DAPO 告诉你“真的在 32B 上跑时，哪些细节会让训练从 30 分爬到 50 分”。

1. 论文结构地图

原文结构很清晰：

1. Abstract 和 Figure 1: 主结论，DAPO 在 Qwen2.5-32B base 上 AIME 2024 达到 50 分。
2. Introduction: 说明 o1/R1 之后社区复现困难，naive GRPO 只有 30 分，问题包括 entropy collapse、reward noise、training instability。
3. Preliminary: 回顾 PPO、GRPO、为什么去掉 KL penalty、为什么用 rule-based reward。
4. Section 3 DAPO: 论文主菜，定义 DAPO objective 和四个 key techniques。
5. Section 3.1 Clip-Higher: 解释为什么 PPO/GRPO 的上裁剪会压住低概率探索 token。
6. Section 3.2 Dynamic Sampling: 解释全对/全错 prompt 为什么没有有效梯度。
7. Section 3.3 Token-Level Policy Gradient Loss: 解释 sample-level loss 在长 CoT 里如何改变长 response 的权重。
8. Section 3.4 Overlong Reward Shaping: 解释截断样本的 reward noise，以及 soft overlong punishment。
9. Section 3.5 Dataset Transformation: 说明为什么把数学答案改成整数，以便 rule reward 稳定解析。
10. Section 4 Experiments: 训练细节、主结果、逐步 ablation、训练动态监控和 case study。
11. Appendix A: 展示数学题 answer transformation 的例子。

如果只读一遍，优先看 Figure 1、Figure 2、Figure 3、Figure 4、Figure 5、Figure 6、Table 1，以及 Algorithm 1。它们分别回答“结果多强”“entropy 怎么塌”“哪些 prompt 没梯度”“token-level loss 改了些什么”“截断惩罚为什么吵”“dynamic sampling 是否拖慢”“每个 trick 贡献多少”。

2. 背景: R1 之后，大家卡在哪里

DeepSeek-R1 的故事是：只用可验证奖励做 RL，模型可以在数学推理里自发出现反思、回溯、验证等长 CoT 行为。但 R1 技术报告没有公开所有训练细节。很多团队照着 GRPO/RLVR 的大方向做复现，发现一旦跑到长 CoT 和大模型，训练经常不稳定。

DAPO 论文里给了一个具体起点：他们用 Qwen2.5-32B base model 做 initial GRPO run，只达到 AIME 30 分，明显低于 DeepSeek-R1-Zero-Qwen-32B 的 47 分。作者进一步分析，naive GRPO 会遇到：

- entropy collapse: 采样 response 越来越相似，模型过早确定化。
- reward noise: 尤其是过长截断样本，被错误地当成坏推理。
- training instability: 长 CoT 下 length、entropy、reward、validation accuracy 互相牵动。

所以 DAPO 的贡献不是把 RLVR 从 0 发明出来，而是把“为什么看起来正确的 GRPO 会在大规模 reasoning RL 上跑不动”拆成可操作问题。它是一篇 recipe paper，也是一篇 reproducibility paper。

3. 先分清 PPO、GRPO、DAPO

PPO 的核心是 importance ratio clipping。设旧策略生成 token 的概率是 $\pi_{\text{old}}(\text{token})$ ，新策略概率是 $\pi_{\text{new}}(\text{token})$ ，则：

$$\text{ratio}_t = \pi_{\text{new}}(\text{token}_t \mid \text{prefix}) / \pi_{\text{old}}(\text{token}_t \mid \text{prefix})$$

PPO 不希望新策略一步走太远，所以把 ratio 限制在：

$$[1 - \epsilon, 1 + \epsilon]$$

GRPO 相比 PPO 的关键变化是去掉 value model，用同一个 prompt 下的一组 responses 的相对 reward 来估计 advantage。对某个 prompt 采样 G 条 response，reward 是 R_i ，则：

$$\text{adv}_i = (R_i - \text{mean}(R_1 \dots R_G)) / \text{std}(R_1 \dots R_G)$$

在数学 RLVR 里，reward 通常来自最终答案是否可验证正确。DAPO 进一步做了几件事：

- 去掉 KL penalty，因为 long-CoT reasoning model 允许离初始 base distribution 更远。
- 使用 rule-based reward，正确给 +1，错误给 -1，避免 learned RM reward hacking。
- 把 PPO/GRPO 的对称 clip 改成上下不对称 clip。
- 把 batch 采样改成只保留有正确也有错误的 prompt group。
- 把 sample-level loss 改成 token-level loss。
- 对过长截断样本加线性 soft penalty，而不是粗暴惩罚。

一句话：DAPO 是围绕 GRPO 做的一组长 CoT RL 稳定化改造。

4. DAPO 总流程

论文 Algorithm 1 可以读成下面这张图：

```
initial policy  $\pi_{\theta}$ 
task prompts  $D$  with integer answers
    |
    v
for each RL step:
    |
    v
sample prompt batch
    |
    v
copy current policy to  $\pi_{\text{old}}$ 
    |
    v
for each prompt, sample  $G$  responses from  $\pi_{\text{old}}$ 
    |
    v
compute rule rewards by answer equivalence
    |
    v
dynamic sampling filter:
keep groups with at least one correct and one incorrect
    |
    v
compute group-relative advantages
```

```

      |
      v
optimize DAPO clipped objective for several iterations
      |
      v
updated policy pi_theta

```

Dynamic sampling 是这个图里最容易被忽略的环节。它发生在训练更新之前，不是在 loss 里加一个小项。也就是说 DAPO 改了数据进入 optimizer 的方式。

5. Rule-based reward 和答案整数化

DAPO 使用 rule-based reward:

```

reward = +1 if predicted_answer is equivalent to ground_truth
reward = -1 otherwise

```

这和上一专题 PRM 有明显区别。PRM 是过程奖励，检查每一步；DAPO 主要使用最终答案可验证的 outcome reward。为什么这样做仍然有效？因为数学竞赛题有相对可靠的答案判定，可以避免 learned reward model 的 reward hacking。

但数学答案格式很复杂，可能是表达式、根式、分数、公式。解析错了，reward 就会错，RL 会被噪声带偏。DAPO 的数据处理因此很重要：作者从 web 和官方竞赛页面收集数据，再把答案转成更容易 parse 的整数形式。例如原答案如果是 $a + c * \sqrt{b}$ ，就改写问题让最终回答变成 $a + b + c$ 。

最终得到 DAPO-Math-17K，包含约 17K prompts，每个 prompt 配一个整数答案。这个细节不是边角料，它直接服务于 RLVR 的可靠性。Rule reward 越稳定，RL 越像在优化推理；rule reward 越脏，RL 越像在放大奖励解析器的 bug。

6. DAPO objective 的张量形状

设一个 rollout batch 有:

```

B prompts
G responses per prompt
T_i tokens in response i

```

常见张量可以想成:

```

log_probs_old shape: [B * G, max_T]
log_probs_new shape: [B * G, max_T]
response_mask shape: [B * G, max_T]
rewards shape: [B, G]
advantages shape: [B, G]

```

每个 token 的 ratio 是:

```
ratio_i_t = exp(log_prob_new_i_t - log_prob_old_i_t)
```

每条 response 的 advantage 来自同 prompt 的 group reward:

```
adv_i = (reward_i - group_mean) / group_std
```

然后把 adv_i broadcast 到这条 response 的所有 token:

```
adv_token_i_t = adv_i
```

DAPO 的核心 loss 单元仍然是 PPO-style clipped surrogate:

```
loss_i_t = - min(
    ratio_i_t * adv_i,
    clip(ratio_i_t, 1 - epsilon_low, 1 + epsilon_high) * adv_i
)
```

区别在于:

- epsilon_low 和 epsilon_high 不相等。
- 只训练 mixed correctness 的 prompt group。
- token-level loss 用所有有效 token 聚合。
- reward 里可能加入 overlong length penalty。

7. Trick 1: Clip-Higher

Naive PPO/GRPO 常用对称裁剪, 比如 `epsilon = 0.2`:

```
ratio in [0.8, 1.2]
```

当 advantage 为正时, 我们希望提高好 token 的概率。问题是, 对低概率探索 token 来说, 1.2 倍太小。例如旧概率是 0.01, 上裁剪后最多到 0.012; 旧概率是 0.9, 上裁剪后理论上到 1.08, 但概率本来就不能超过 1。结果是高概率 exploitation token 几乎不受影响, 低概率 exploration token 却被卡得很死。

这会让模型过早走向确定化, entropy collapse。论文 Figure 2 显示, 使用 Clip-Higher 后, AIME accuracy 更好, actor model generation entropy 也维持得更健康。Figure 3a 进一步支持这个解释: 被 up-clipped 的 token 平均概率偏低, 说明上裁剪确实主要限制了低概率探索 token。

DAPO 的做法是:

```
ratio in [1 - epsilon_low, 1 + epsilon_high]
epsilon_low = 0.2
epsilon_high = 0.28
```

为什么只抬高 upper clip, 而不扩大 lower clip? 论文解释是, 如果降低下界约束太多, 会更容易把某些 token 概率压到 0, 反而让采样空间塌掉。Clip-Higher 的精神是 “给正 advantage 的探索 token 更多上升空间, 但不要鼓励更激烈地杀死 token”。

本仓库对应函数是 `asymmetric_clip_loss`:

```
def asymmetric_clip_loss(
    log_probs_new,
    log_probs_old,
    advantages,
    response_mask,
    eps_low=0.2,
    eps_high=0.28,
):
    ratio = (log_probs_new - log_probs_old).exp()
    A = advantages.unsqueeze(1)
    surr1 = ratio * A
    surr2 = ratio.clamp(1 - eps_low, 1 + eps_high) * A
    return -torch.min(surr1, surr2)
```

学习时可以构造一个 positive advantage、ratio 大约 1.28 的例子。对称 clip 会截到 1.2, Clip-Higher 会允许到 1.28, loss 更小, 说明这次更新被允许更充分地提升好 token。

8. Trick 2: Dynamic Sampling

GRPO 的 group advantage 有一个很直接的问题: 如果同一个 prompt 的 G 条 response 全对, 所有 reward 相同, advantage 全为 0; 如果全错, 也一样 advantage 全为 0。

```
rewards = [1, 1, 1, 1]      -> std = 0 or advantage = 0
rewards = [-1, -1, -1, -1] -> std = 0 or advantage = 0
rewards = [1, -1, 1, -1]    -> useful gradient
```

论文 Figure 3b 显示, 训练过程中 avg@32 为 100% 的 prompt 比例会增加。这意味着 batch 里越来越多 prompt group 没有有效梯度。它们还会占显存、占采样和训练 slot, 让 batch gradient 更小、更吵。

DAPO 的做法是 oversample and filter:

```
keep prompt group only if:
0 < number_of_correct_responses < G
```

也就是只保留“有对有错”的 prompt。这样每个进入训练 buffer 的 prompt 都能提供 group-relative signal。

这个策略看起来会增加采样成本, 但论文指出, 在同步 RL 系统里 generation 时间通常被长尾样本支配; 过滤掉 zero-gradient prompts 不一定显著拖慢整体训练。Figure 6 显示, 在 baseline setting 中, dynamic sampling 甚至能更快达到同等性能, 因为有效训练步更多。

本仓库 is_group_useful 已按论文语义实现, 兼容 +1/-1 和 1/0 reward:

```
def is_group_useful(rewards):
    correct = rewards > 0
    return bool(correct.any() and (~correct).any())
```

这就是 Dynamic Sampling 的最小逻辑。

9. Trick 3: Token-Level Policy Gradient Loss

原始 GRPO 常见聚合方式是 sample-level:

```
for each response:
    response_loss = mean(token_losses in this response)
final_loss = mean(response_loss over responses)
```

这意味着每条 response 权重相同。短 response 有 100 个 token, 长 response 有 4000 个 token, 它们对 final loss 的权重仍然一样。于是长 response 里的每个 token 权重更低。

在 long-CoT RL 里, 这会造成两个问题:

1. 高质量长推理里的关键 token 学得不够。
2. 低质量长输出里的重复、乱码、无意义探索也惩罚不够。

论文 Figure 4 展示了 token-level loss 对 entropy 和 mean response length 的影响。作者的解释是, token-level loss 让每个 token 平等参与梯度更新, 而不是让每条 response 平等。这样长 response 的模式如果提高 reward, 就会被充分强化; 如果降低 reward, 也会被充分压制。

Token-level 聚合是:

```
final_loss = sum(valid_token_loss) / number_of_valid_tokens
```

本仓库对比很清楚:

```
def token_level_loss(per_token_loss, response_mask):
    return (per_token_loss * response_mask).sum() / response_mask.sum().clamp(min=1)

def response_level_loss(per_token_loss, response_mask):
    per_resp_loss = (per_token_loss * response_mask).sum(dim=1) / \
        response_mask.sum(dim=1).clamp(min=1)
    return per_resp_loss.mean()
```

这段代码是 DAPO 最值得亲手算一遍的地方。你可以设置一个短 response loss 高、长 response loss 低的 toy case，看两种聚合给出的 final loss 如何不同。

10. Trick 4: Overlong Reward Shaping

长 CoT RL 必须设置最大生成长度。问题是，如果一个 response 因为超过最大长度被截断，它的最终答案可能根本没有生成出来。默认把这种样本直接给惩罚，会带来 reward noise: 一条推理过程也许方向正确，只是太长，却被当作完全错误。

DAPO 先试了 Overlong Filtering: 对截断样本 mask 掉 loss。Figure 5 显示这能明显稳定训练并提升表现。

进一步，论文提出 Soft Overlong Punishment。设:

```
expected_len = Lmax - Lcache
max_len = Lmax
```

长度惩罚是:

```
if len <= expected_len:
    penalty = 0
elif expected_len < len <= max_len:
    penalty = (expected_len - len) / Lcache
else:
    penalty = -1
```

```
shaped_reward = rule_reward + penalty
```

所以在 cache 区间内，越接近 max_len 惩罚越大；超过 max_len 后直接加 -1。这个设计比“只要截断就判死刑”更平滑，也给模型一个逐渐缩短过长 response 的信号。

论文训练设置里:

```
expected maximum length = 16,384 tokens
soft punish cache = 4,096 tokens
generation max length = 20,480 tokens
```

本仓库 overlong_shaping 实现了这个线性版本:

```
def overlong_shaping(rewards, response_lens,
                    expected_len=16384, cache_len=4096):
    max_len = expected_len + cache_len
    penalty = torch.zeros_like(rewards, dtype=torch.float32)
    in_cache = (response_lens > expected_len) & (response_lens <= max_len)
    penalty[in_cache] = (expected_len - response_lens[in_cache]) / cache_len
    penalty[response_lens > max_len] = -1.0
    return rewards.float() + penalty
```

这里要注意: penalty 是加到 rule reward 上, 不是把正 reward 乘一个衰减因子。对错误且过长的 response, 它会更负; 对正确但过长的 response, 它会被扣分。

11. 四个 trick 的证据链

Table 1 是 DAPO 论文最重要的 ablation。它不是一次性把所有 trick 打开, 而是逐步加:

DeepSeek-R1-Zero-Qwen-32B: 47
Naive GRPO: 30
+ Overlong Filtering: 36
+ Clip-Higher: 38
+ Soft Overlong Punishment: 41
+ Token-level Loss: 42
+ Dynamic Sampling, DAPO full: 50

读这个表要注意三点:

第一, naive GRPO 到 30 分, 说明 base model + GRPO + rule reward 并不是自动成功。缺 recipe 时, 大规模 RLVR 远低于 R1 参考结果。

第二, Overlong Filtering 和 Soft Overlong Punishment 的合计贡献很大, 说明 reward noise 在长 CoT 里不是小问题。错误的截断惩罚会直接破坏训练。

第三, Dynamic Sampling 最后从 42 拉到 50, 说明有效 batch 质量很关键。采样更多但训练更有效, 可能比 “每个 prompt 固定采 G 条然后全用” 更划算。

这条证据链支持论文主张: DAPO 的贡献是 recipe 组合, 而不是单个 trick 单独创造奇迹。

12. Training details: 这些数字要记住

论文的训练设置给得比较具体:

- Base model: Qwen2.5-32B。
- Training framework: verl。
- Baseline: naive GRPO, group reward normalization。
- Optimizer: AdamW。
- Learning rate: constant 1e-6。
- Warm-up: 20 rollout steps。
- Rollout prompt batch size: 512。
- Responses per prompt: 16。
- Training mini-batch size: 512。
- Gradient updates per rollout step: 16。
- Clip-Higher: $\epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$ 。
- Expected response length: 16,384 tokens。
- Soft punish cache: 4,096 tokens。
- Generation max length: 20,480 tokens。
- AIME evaluation: repeat evaluation set 32 times and report avg@32。
- Evaluation inference: temperature 1.0, top_p 0.7。

这些数字不是让你背, 而是让你理解 DAPO 是系统 recipe。比如 responses per prompt = 16 直接决定 Dynamic Sampling 的 group 粒度; max length = 20,480 直接决定长 CoT 的显存、吞吐和截断噪声; avg@32 说明 reasoning model 的评估本身也需要采样稳定性。

13. 训练动态: DAPO 让你监控什么

Section 4.3 很值得读。作者说大规模 LLM RL 是复杂系统工程，单个子系统的小改动会通过迭代 RL 放大。论文建议监控几个中间指标：

1. Mean response length: 长度上升通常表示模型探索更复杂推理，但长度不是越长越好。停滞或下降可能是训练恶化信号。
2. Reward score: 训练集 reward 可能稳定升高，但不一定和 validation accuracy 强相关，这意味着可能过拟合训练题。
3. Generation entropy: 太低代表探索塌缩，太高可能代表乱码、重复和过度探索。
4. Mean probability: 与 entropy 方向相反，可以辅助判断分布是否过尖。

Figure 7 展示了这些指标随训练步变化。对学习者来说，这部分特别重要，因为真实 RL 训练不是只看 final AIME 分数。等 final 分数掉了再查，已经太晚了；你要在 length、entropy、reward 曲线上提前发现不对劲。

14. Case study: 反思行为不是硬编码的

Section 4.4 观察到一个有意思的现象：随着 RL 训练推进，模型推理模式会演化。早期几乎没有对前面步骤的检查和反思，后期开始出现 reflection、backtracking、重新考虑几何角度等行为。

Table 2 展示了一个四面体几何题的例子，模型先按坐标思路推进，然后出现类似 “wait a moment, rethink” 的反思行为。论文没有把这当作严格机制证明，而是作为 case study 说明 RL 可以强化既有有用模式，也可能诱导出训练初期少见的新推理模式。

这个现象和 DeepSeek-R1 的 aha moment 叙事相呼应。但你要保持清醒：case study 是帮助理解，不是主证据。主证据仍然是 Figure 1、Table 1 和各 trick 的 ablation。

15. DAPO 和 PRM/RLVR 的关系

上一篇 Let's Verify Step by Step 强调 process supervision，训练 PRM 去检查每一步。DAPO 这篇主要走 RLVR outcome reward 路线，用最终答案等价性给 reward。

它们不是互相否定：

- PRM 解决过程信用分配，但需要昂贵或自动化的 step-level labels。
- RLVR reward 便宜、客观、可扩展，但只看最终答案，credit assignment 依赖 policy optimization 和采样。
- DAPO 的四个 trick 本质上是在让 outcome reward 更稳定地驱动长 CoT policy。
- 后续系统可以把 PRM、GenRM、verifier、tool checking 和 DAPO-style RL recipe 组合起来。

所以学习路线是：PRM 告诉你为什么过程信号重要；DAPO 告诉你即使只有 outcome reward，也可以通过正确的 RL recipe 在可验证任务上诱发复杂推理。

16. 和后续 VAPO、Dr.GRPO、GenRM 的关系

本专题还放了几个后续方向的最小代码：

vapo_minimal.py 讲 Length-Adaptive GAE。它关注 value/advantage 的时间信用分配，让 response 长度影响 GAE 的 lambda。

dr_grpo.py 讲 Dr.GRPO，用 MAD 替代 std 做 group normalization，并加入 length penalty，目标是减轻 outlier 和长度偏置。

genrm.py 讲 Generative Reward Model, 让 LLM 生成 critique 和 score, 而不是直接用 scalar head 输出 reward。它更可解释, 但推理成本更高。

capstone_dapo_ablation.py 是本地消融 mock, 把 DAPO 四件套拆开看 accuracy、length、aha frequency 的变化。

这些后续代码不都是 DAPO 原论文内容, 但它们放在同一专题里是合理的: DAPO 之后, 社区继续围绕 long-CoT RL 的 credit assignment、length bias、reward modeling 和 training stability 修补。

17. 本仓库代码地图

本章最重要文件:

- learning/rl-sota-2026/src/dapo_minimal.py: DAPO 四件套的最小实现。
- learning/rl-sota-2026/src/tests/test_dapo_each_trick.py: 每个 trick 的独立测试。
- learning/rl-sota-2026/src/capstone_dapo_ablation.py: DAPO trick 组合消融 mock。
- learning/rl-sota-2026/src/vapo_minimal.py: VAPO length-adaptive GAE。
- learning/rl-sota-2026/src/dr_grpo.py: Dr.GRPO advantage normalization。
- learning/rl-sota-2026/src/genrm.py: Generative RM demo。
- learning/rl-sota-2026/lectures/01-dapo-four-tricks.md: DAPO lecture。
- learning/rl-sota-2026/lectures/12-capstone-dapo-on-rl.md: capstone 训练计划。

建议先跑:

```
python learning\rl-sota-2026\src\dapo_minimal.py
python learning\rl-sota-2026\src\capstone_dapo_ablation.py
$env:PYTEST_DISABLE_PLUGIN_AUTOLOAD='1'; python learning\rl-sota-2026\src\tests\test_dapo_each_tr
```

如果你只读代码不读论文, 很容易以为这是四个随意 trick。读完论文后再看代码, 你会发现每个函数都对应一个明确训练病灶。

18. 30-60 分钟本地实验

实验目标: 亲手理解 Dynamic Sampling 和 Token-Level Loss 为什么重要。

实验 A: Dynamic Sampling

1. 在 Python 里构造三组 rewards:
[1, 1, 1, 1]
[-1, -1, -1, -1]
[1, -1, 1, -1]
2. 调用 is_group_useful。
3. 观察只有第三组返回 True。
4. 自己解释:
为什么全对 prompt 看起来很好, 却对 GRPO 没有 group-relative gradient?

实验 B: Token-Level vs Response-Level

1. 构造两个 response:
response A 长度 2, 每 token loss = 2
response B 长度 4, 每 token loss = 1

2. 分别计算 `token_level_loss` 和 `response_level_loss`。
3. 解释为什么 `response-level` 会让每条 `response` 同权, `token-level` 会让每个 `token` 同权。
4. 把这个结果对应到长 CoT:
长 `response` 中的重复模式为什么需要被充分惩罚?

实验 C: Overlong Reward Shaping

1. 设置 `expected_len = 4096`, `cache_len = 200`。
2. `rewards = [1, 1, 1]`
3. `lengths = [3000, 4196, 10000]`
4. 观察 shaped rewards:
短样本不变。
cache 区间线性扣分。
超过 `max_len` 加 `-1` penalty。

这三个实验足以把 DAPO 从“论文名词”变成“你手里能调的训练开关”。

19. AI agent 应该怎样辅助你学 DAPO

这篇论文非常适合让 agent 当训练 debug 教练。不要让 agent 泛泛总结四个 trick, 而要让它把每个 trick 绑定到失败曲线和代码函数。

推荐提示词:

我正在学 DAPO。请你按训练故障诊断的方式考我。

一次只问一个问题。

每个问题必须包含:

1. 一个训练症状, 例如 `entropy collapse`、全对 prompt 变多、长度爆炸、截断噪声。
2. 让我判断应该用 DAPO 哪个 trick。
3. 让我指出本仓库哪个函数对应这个 trick。
4. 让我解释它改变了哪个张量或 `loss reduction`。

如果我回答只停留在名词层面, 请继续追问公式和 `shape`。

更好的用法是让 agent 生成 toy tensor:

请构造一个 `B=2`, `G=4` 的 `reward tensor`,

其中一个 prompt 全对, 一个 prompt 有对有错。

让我手算 GRPO advantage, 再解释 `Dynamic Sampling` 会保留哪一个。

你自己必须动手算一遍。Agent 的价值是出题、纠错、把论文和代码对齐, 不是替你把摘要背下来。

20. 常见误读

误读一: DAPO 发明了 RLVR。

更准确: RLVR 在 R1、DeepSeekMath、定理证明和代码任务里已经很重要。DAPO 的贡献是公开可复现的大规模 long-CoT RL recipe。

误读二: Clip-Higher 就是把 PPO clip 调大。

更准确: 它是上下裁剪解耦, 只提高 upper clip, 保留下载剪约束。动机是给低概率探索 token 更多上升空间, 同时避免概率被过度压低。

误读三: Dynamic Sampling 是为了让 batch 题更难。

更准确: 它是为了过滤 group advantage 为 0 的 prompt。全对和全错都没有相对学习信号。

误读四: Token-level loss 一定鼓励更长输出。

更准确: 它让每个 token 同权。高质量长推理可以学到, 低质量长乱码也会被更充分惩罚。论文认为这让长度增长更健康。

误读五: Overlong Reward Shaping 是惩罚所有长回答。

更准确: 它只在 expected length 后逐渐加 penalty, 超过最大长度后加 -1。它的目标是减少截断 reward noise, 而不是禁止长推理。

误读六: AIME 50 说明 DAPO 已经解决所有 reasoning RL。

更准确: 论文主要在数学任务上验证, 且依赖可验证整数答案、Qwen2.5-32B、verl 工程和特定 hyper-parameters。迁移到代码、agent、开放问答还需要重新验证。

21. 现代意义

DAPO 的现代意义有三层。

第一, 它把 R1 之后的“神秘 RL recipe”具体化。社区不再只能看最终技术报告, 而能看到 clip、sampling、loss reduction、length reward、dataset transformation、monitoring metrics 这些工程细节。

第二, 它提醒你 reasoning RL 是系统工程。一个看似小的 loss reduction 改法, 会影响 entropy、length、reward、validation accuracy 和训练吞吐。单看公式很容易低估这些耦合。

第三, 它给后续开源 reasoning RL 提供了基线。你可以在 DAPO 之上继续研究 value model、PRM、GenRM、Dr.GRPO、VAPO、tool reward、多任务 RL 和 agent reward, 但你需要先能解释 DAPO 为什么把 naive GRPO 从 30 拉到 50。

22. 闭卷掌握检查

读完后你应该能闭卷回答:

1. DAPO 为什么说 naive GRPO 只有 AIME 30 分, 这说明了什么。
2. DAPO 为什么去掉 KL penalty, 这和普通 RLHF 的目标有什么差异。
3. Rule-based reward 的 $+1/-1$ 公式是什么, 为什么答案整数化重要。
4. Clip-Higher 为什么主要帮助低概率 exploration token。
5. 为什么 DAPO 不对称地设置 $\epsilon_{\text{low}} = 0.2, \epsilon_{\text{high}} = 0.28$ 。
6. Dynamic Sampling 为什么要过滤全对和全错 prompt。
7. Token-level loss 和 response-level loss 的 denominator 分别是什么。
8. Overlong Reward Shaping 的三段 penalty 公式是什么。
9. Table 1 中 naive GRPO、Overlong Filtering、Clip-Higher、Soft Overlong、Token-level、Dynamic Sampling 各自的分数变化说明了什么。
10. Figure 7 中 length、reward、entropy、mean probability 分别用于监控什么风险。
11. 在 `dapo_minimal.py` 中, 哪个函数实现 Clip-Higher, 哪个函数实现 Dynamic Sampling, 哪个函数实现 Overlong Shaping。
12. 如果训练 reward 上升但 validation AIME 不升, 你会优先检查哪些曲线和数据问题。

真正掌握的标志是: 你能把 DAPO 画成一个 RL 训练系统图, 并能用 toy tensor 解释四个 trick 分别改变了采样、ratio clip、loss aggregation 和 reward shaping 的哪一段。